Functions of the Network Layer

The Network Layer (Layer 3 of the OSI model) performs **two main functions**:

1. Forwarding (Data Plane Function)

Forwarding is the process of **moving a packet from a router's input port to the correct output port** so it can continue its journey toward the destination.

Key Points:

- Happens on every router the packet passes through.
- Uses a forwarding table / flow table.
- Must be fast, because it is performed for every packet.
- Based on the destination IP address (in traditional IP networks).

Forwarding Actions

In SDN (match–action):

- forward the packet to an output port
- drop the packet
- replicate the packet
- rewrite header fields

Forwarding Table / Flow Table

- Installed in routers.
- Used by the data plane to forward packets.
- Flow table (generalized forwarding) can:
    - forward
    - drop
    - replicate
    - rewrite header fields (L2/L3/L4)

2. Routing (Control Plane Function)

Routing is the process of **computing and selecting the best path** for packets to travel from source to destination.

Key Points:

- Happens **on routers (per-router control)** or **in a central controller (SDN)**.
- Must maintain and update routing tables.
- Uses routing algorithms (e.g., Dijkstra, Bellman-Ford).
- Uses routing protocols (OSPF, RIP, BGP).

Routing Responsibilities

- Discovering network topology
- Calculating least-cost paths
- Updating routes when the topology changes
- Exchanging routing information

Two Approaches for Control Plane

1. Per-Router Control (Traditional Internet)

- Every router runs a **routing algorithm locally**.
- Routers exchange routing information with neighbors.
- Each router computes its own forwarding table.
- Examples:
    - **OSPF** (Link-State)
    - **BGP** (Path Vector)

Key point: Routers directly interact with each other.

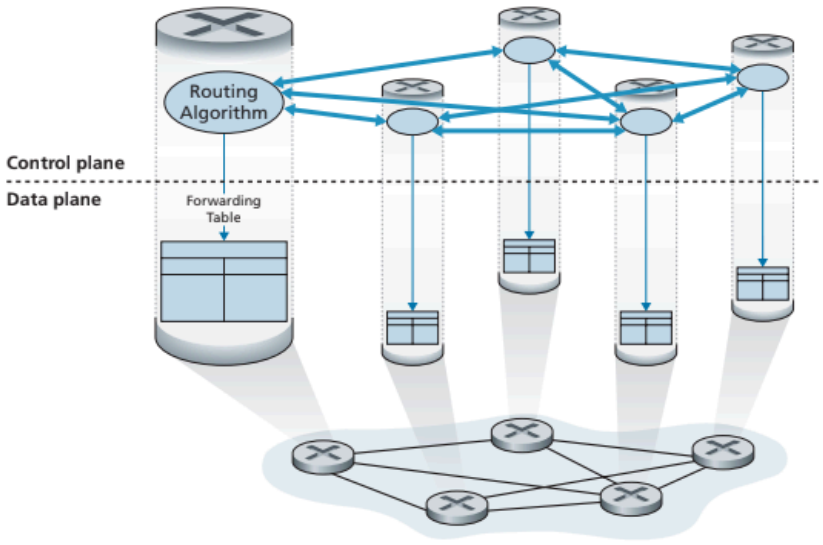


Figure 5.1 ♦ Per-router control: Individual routing algorithm components interact in the control plane

2. Logically Centralized Control (SDN)

- A central controller computes routing/flow tables.
- Routers contain only a simple **Control Agent (CA)**.
- CAs do NOT run routing algorithms; they only listen to controller commands.
- Uses protocols like **OpenFlow**.

Real-world examples:

- Google B4
- Microsoft SWAN
- Comcast ActiveCore
- Deutsche Telekom Access 4.0
- 4G/5G core networks
- China Telecom, China Unicom

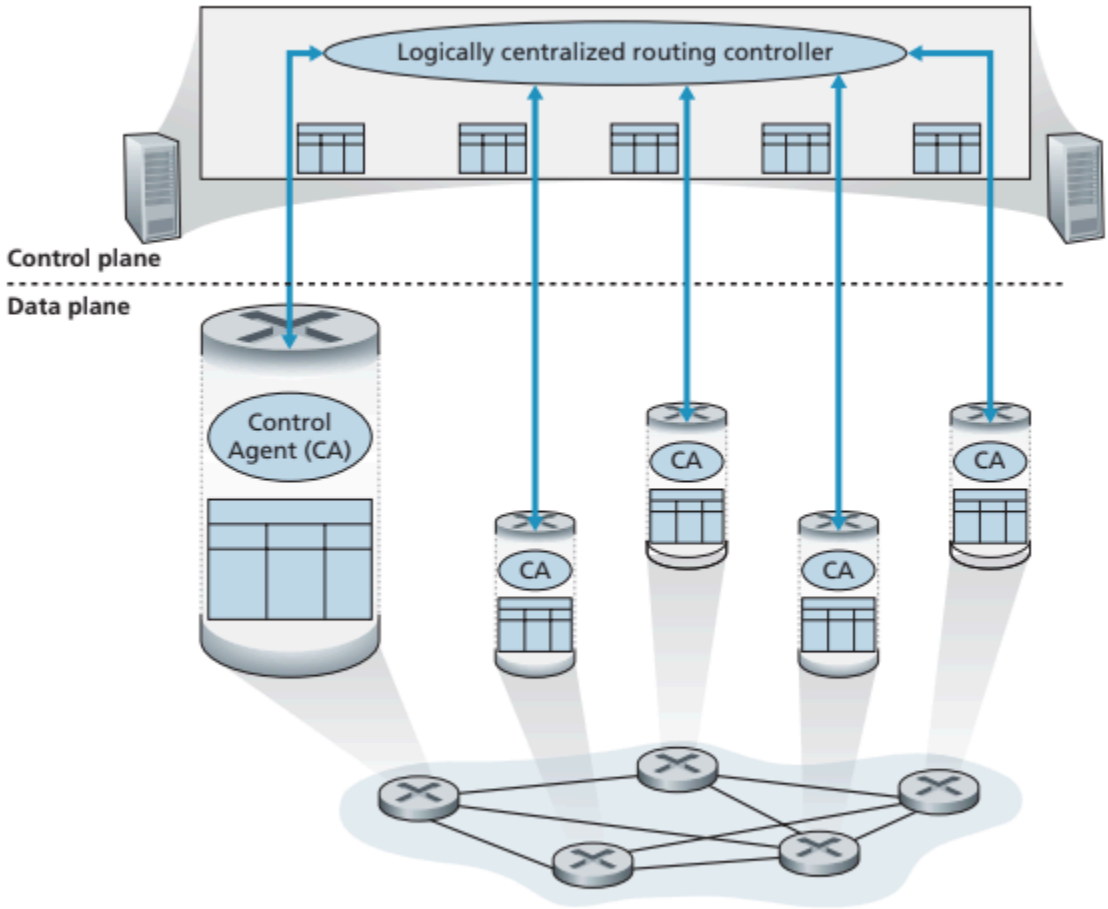


Figure 5.2 ♦ Logically centralized control: A distinct, typically remote, controller interacts with local control agents (CAs)

5.2 Routing Algorithms

Goal: Compute **least-cost (best) paths** between source–destination router pairs.

Graph Model

- Network = **Graph G(N, E)**
  - **N** = routers
  - **E** = physical links

- Each edge has a **cost c(x, y)**:
  - distance
  - speed
  - monetary cost
  - arbitrary metric set by admin

Path Cost: Sum of all link costs on the path.

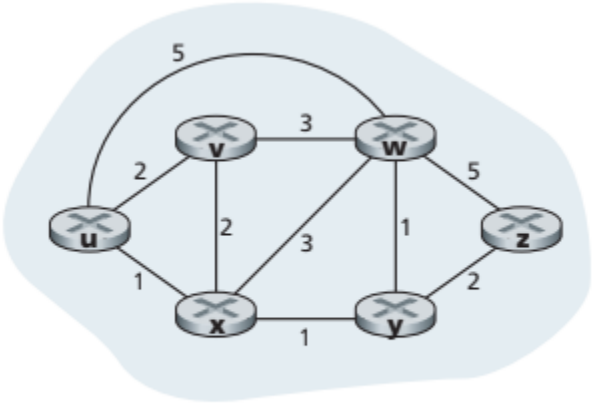


Figure 5.3 ♦ Abstract graph model of a computer network

Classification of Routing Algorithms

1. Centralized vs Decentralized

Centralized (Link-State)

- Requires **global knowledge** of network.
- Algorithm runs at one (or all) routers but with full network info.
- Example: **Dijkstra's Algorithm** (used in OSPF).

Decentralized (Distance-Vector)

- No router has global information.
- Router knows only:
    - its neighbors
    - cost of links to neighbors
- Routers iteratively exchange information.
- Example: **Bellman-Ford** (used in RIP).

2. Static vs Dynamic

Static Routing

- Rarely changes.
- Manually configured.

Dynamic Routing

- Updates paths automatically based on changes.
- More responsive but may have:
    - routing loops
    - oscillations
    - convergence delays

3. Load-Sensitive vs Load-Insensitive

Load-Sensitive

- Link costs depend on congestion.
- Early ARPANET used this (but caused instability).

Load-Insensitive (Modern Internet)

- OSPF, RIP, BGP use fixed link costs.
- Do NOT measure congestion directly.

A. Short-Concept Questions — Answers

1. What is the difference between the data plane and the control plane?

- **Data Plane:** Handles actual packet forwarding (router → router). Uses the forwarding/flow table.

- **Control Plane:** Computes and installs the forwarding rules (routing algorithms, SDN controller).

2. What is the main function of the flow table in SDN?

The flow table stores **match–action rules** that determine:

- where a packet is forwarded, OR
- whether it is dropped, replicated, or has headers rewritten.

3. Compare per-router control with logically centralized control.

Per-Router Control	Logically Centralized Control (SDN)
Each router runs routing algorithms (OSPF/BGP).	A central controller computes routing.
Routers exchange routing messages with neighbors.	Routers only contain simple Control Agents (CAs).
Fully distributed.	Centralized logic, distributed implementation.
Traditional Internet approach.	Used in SDN (OpenFlow).

4. What is the role of a Control Agent (CA) in SDN?

- A simple software component inside each router/switch.
- Communicates with the SDN controller.
- Installs flow table entries **as instructed**.
- Does *not* compute routes.

5. Define link-state vs distance-vector.

- **Link-State (LS):**
  - Router has **global knowledge** (full network map).
  - Runs Dijkstra's algorithm.
  - Example: OSPF.

- **Distance-Vector (DV):**
  - Router knows only **its neighbors and their distances**.
  - Periodically exchanges distance vectors with neighbors (Bellman-Ford).

- Example: RIP.

6. Why are modern routing algorithms load-insensitive?

Because using congestion-dependent link costs caused **instability, oscillations, and loops** in early networks (like ARPANET).
Therefore, modern protocols (OSPF, RIP, BGP) use **fixed** link costs.

7. What is a "least-cost path"?

The path between a source and destination that has the **minimum total cost**, obtained by adding the costs of all links in the path.

8. What does "logically centralized" mean in SDN?

The control logic **appears** to be in a single controller even though it may actually run on multiple distributed servers for reliability and scalability.

9. Give two examples of real-world SDN deployments.

- Google **B4** WAN
- Microsoft **SWAN**
  (Any two acceptable: Comcast ActiveCore, Deutsche Telekom Access 4.0, China Telecom, China Unicom)

10. What is the difference between static and dynamic routing?

- **Static:** Manually configured, changes rarely, not adaptive.
- **Dynamic:** Automatically updates based on topology changes, uses routing protocols.

Dijikstra Algorithm:

Link-State (LS) Algorithm for Source Node *u*

```
1  Initialization:
2   N' = {u}
3   for all nodes v
4     if v is a neighbor of u
5       then D(v) = c(u,v)
6     else D(v) = ∞
7
8  Loop
9   find w not in N' such that D(w) is a minimum
10  add w to N'
11  update D(v) for each neighbor v of w and not in N':
12        D(v) = min(D(v), D(w)+ c(w,v) )
13   /* new cost to v is either old cost to v or known
14    least path cost to w plus cost from w to v */
15 until N'= N
```

step	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	1,u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	uxyv		3, y			4, y
4	uxyvw					4, y
5	uxyvwz					

Table 5.1 ♦ Running the link-state algorithm on the network in Figure 5.3

A. Complexity of the simple LS (Dijkstra-style) implementation

- If there are **n** nodes (not counting the source) and you pick the next minimum-cost node by scanning the list each iteration, you do:

    - n checks in the 1st iteration, n−1 in the 2nd, …, 1 in the last → total ≈ **n(n+1)/2** comparisons.
- Therefore the worst-case running time is **Θ(n²)**, commonly written **O(n²)**.

Better implementations

- Use a priority queue (heap) to select the minimum instead of linear scan:
    - With a binary heap the complexity becomes **O((n + m) log n)** (where m = number of edges); often simplified to **O(m log n)** for sparse graphs.
    - With a Fibonacci heap you get **O(m + n log n)** (theoretical best for dense use).
- Bottom line: replacing the linear search with a heap reduces the dominant term from quadratic to near linearithmic.

LS Algorithm Oscillation Problem

1️⃣ What is happening?

In Figure 5.5, some nodes (x, y, z) send traffic to destination **w**.

- The **cost** of a link = **amount of traffic on that link**.
  (More traffic → higher delay → higher cost)

This means link costs **change** depending on routing.

Why oscillations happen

Step 1: Initial State

Nodes x, y, z choose paths normally based on current link costs.

Example:

- y thinks going counterclockwise is cheaper → chooses that path.
- x also chooses a path based on current load.

Step 2: LS (Dijkstra) runs again

All routers recalculate the “least-cost path.”

Now y sees:

- Clockwise cost = 1

- Counterclockwise cost = 1 + e (more expensive)

So:

✔ y switches to the clockwise path

Same for x.

This causes traffic to move from one side of the ring to the other.

Step 3: Next LS run

Now that everyone switched paths:

- The clockwise direction becomes overloaded
- The counterclockwise direction becomes empty (cost = 0)

So all nodes (x, y, z) think:

✔ "Counterclockwise is cheaper! Let's go there."

They all switch again → costs reverse.

Result:

⚠️ Nodes keep switching back and forth

Clockwise → counterclockwise → clockwise → …
This repeating switching is called **oscillation**.

It happens because:

- Link costs depend on load
- All routers recompute their routing at the **same time**

Why is this a problem?

- Routes keep changing
- Heavy unstable traffic
- Packets may loop or experience delays
- Network never reaches a stable routing solution

This is very bad in large networks like the Internet.

Preventing Oscillations

The book gives **two solutions**:

✔ Solution 1: Do NOT let link costs depend on traffic

If link costs were *fixed*, this problem would never happen.

But this solution is **unacceptable**, because we WANT routing to avoid congested links.

So we need a realistic solution.

✔ Solution 2 (Better): Do NOT run LS at the same time

If routers run LS at different random times:

- Not all routers will switch simultaneously
- One router updates first → others update later
- This prevents everyone from changing direction together
- System stabilizes automatically

But researchers found something surprising:

⚠️ Routers can synchronize themselves!

Even if they start at different times, eventually they align and run LS at the same moment.
This causes the oscillation again.

✔ Final Solution: Randomize advertisement times

Each router should:

🎲 Randomly delay when it sends Link-State Advertisements (LSAs)

This ensures:

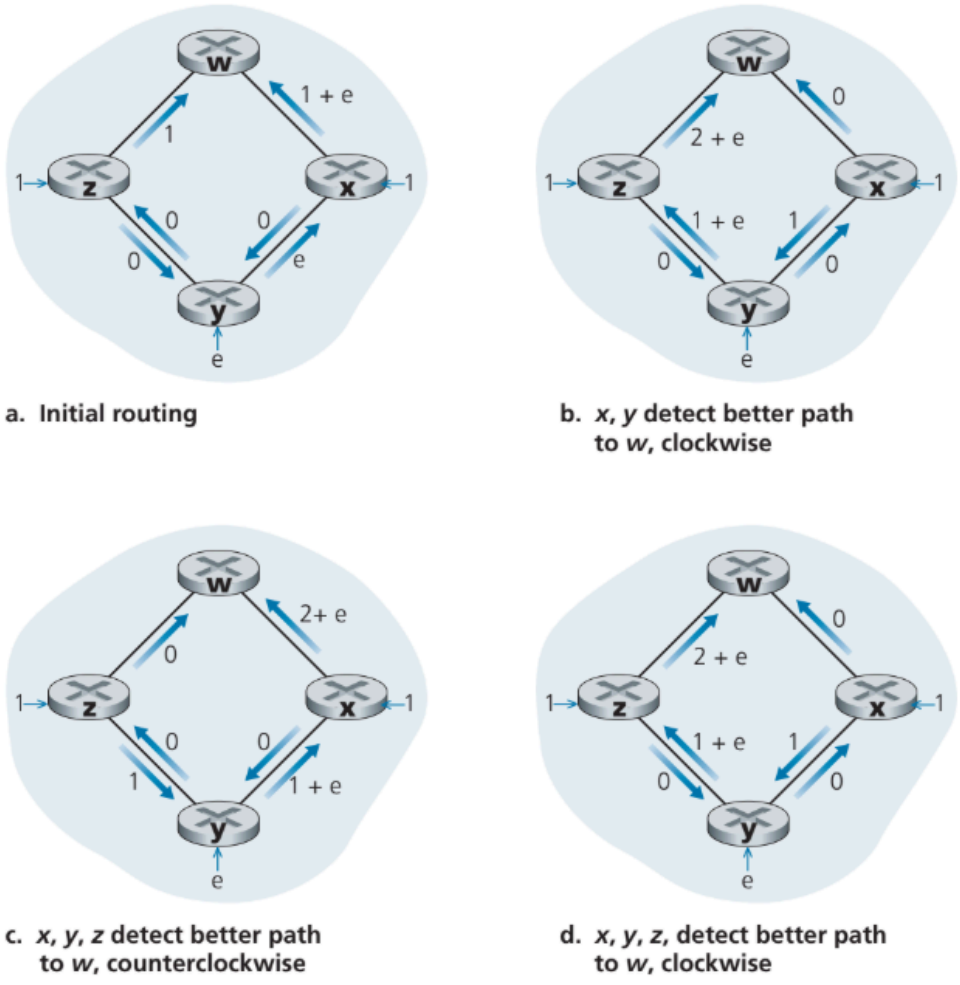
- LS algorithm doesn't run in sync
- Avoids simultaneous route flips
- Routing becomes stable

Final Summary (for exam)

Oscillation occurs because LS routing with load-based link costs makes routers repeatedly switch to cheaper paths at the same time.
This causes traffic to jump back and forth (clockwise ↔ counterclockwise).

To prevent this:

1. **Do not base link cost purely on traffic load** (not practical), OR
2. **Ensure routers do not run LS at the same time**
    - Randomize the time of sending link-state advertisements.



a. Initial routing

b. *x*, *y* detect better path to *w*, clockwise

c. *x*, *y*, *z* detect better path to *w*, counterclockwise

d. *x*, *y*, *z*, detect better path to *w*, clockwise

Figure 5.5 ♦ Oscillations with congestion-sensitive routing

DISTANCE VECTOR (DV) ROUTING – NOTES (Q/A FORMAT)

Q1. What type of algorithm is the Distance Vector (DV) Routing Algorithm?

Answer:
DV is:

- **Iterative** → repeats until no more changes
- **Asynchronous** → nodes do *not* operate in lockstep
- **Distributed** → each node computes using information only from neighbors
- **Self-terminating** → stops automatically when no updates occur

Q2. What information is stored at each node in DV routing?

Answer: Each node x stores:

1. **Cost to each directly connected neighbor**
   → c(x,v)
2. **Its own distance vector**
   → Dx(y)for all destinations y
3. **Distance vectors received from neighbors**
   → Dv(y) for each neighbor v

Q3. What is the Bellman-Ford Equation? Why is it important?

Answer:

$dx(y)=\min_{v \in \text{neighbors of } x} (c(x,v)+dv(y))$
Importance:

- Defines how a node computes the **shortest path** to every destination.
- Forms the **core computation** in Distance Vector routing.
- Determines **next-hop** route (neighbor which gives minimum).

Q4. Explain how the DV update occurs.

Answer:
A node x updates its table when:

1. **A link cost changes**, OR
2. **A neighbor sends a new distance vector**.

Then x:

- Applies Bellman-Ford equation:
  $D_x(y)=\min_v (c(x,v)+D_v(y))$
- If any value changes → sends updated DV to neighbors.

Q5. What triggers sending DV updates?

Answer:

- DV is sent **only when the node's own DV changes**.
- If no change → **no message**.

This makes DV **efficient** and **self-stopping**.

Q6. Why is DV called "asynchronous"?

Answer:
Nodes:

- Do **not** share a global clock
- Receive/send updates at **different times**
- Compute updates **independently**

Thus, updates happen randomly, not simultaneously.

Q7. What is the drawback of DV algorithms?

Answer:
Main drawback: **count-to-infinity problem**.

Reason:

- Nodes may slowly increase cost values if a route is lost
- Bad news travels slowly
- Loops can form temporarily

Example: If a link fails, neighboring nodes keep advertising increasing path costs.

Q8. How does the DV algorithm differ from the LS (Link-State) algorithm?

Feature	DV	LS
Information	Only neighbor DVs	Full network topology
Computation	Bellman-Ford eq	Dijkstra's algorithm
Complexity	Lower	Higher
Messages	Distance vectors periodically or on change	Link-state advertisements to all nodes
Convergence	Slower	Fast
Issues	Count-to-infinity	Oscillations possible

Q9. What is the computational complexity of the DV algorithm?

Answer:
For each update:

- DV runs Bellman-Ford once per destination
- For each destination → checks all neighbors

If a node has:

- **n destinations**
- **k neighbors**

Then:

O(nk)

Multiple iterations happen until convergence → **overall complexity depends on network events**, not fixed.

Q10. Why is the DV algorithm called "self-terminating"?

Answer:
Because:

- Nodes only send updates when their DV changes.
- Eventually, all DVs stabilize.

- No further updates cause no further computation.

Hence, algorithm ends **without any explicit stop signal**.

Q11. Write the Distance Vector algorithm steps.

Answer:

Initialization

- Dx(y)=c(x,y) (∞ if not neighbor)
- Receive DVs from neighbors

Repeat

- Wait for:
    - Link cost change, OR
    - New DV from neighbor
- Update:
  Dx(y)=minv(c(x,v)+Dv(y))
- If any value changes → send DV to neighbors

End

When no more DV changes.

Q12. Give an example of DV table update (short explanation).

Answer:
If x receives D_y(z) = 4 and link cost c(x,y)=2:

New possible cost to z through y is:

2+4=6

If this is better than current Dx(z), update Dx(z)=6 and send DV to neighbors.

Q13. What causes routing loops in DV?

Answer:
Loops happen when:

- Nodes have **outdated information**,

- A link fails or cost increases,
- Neighbors continue advertising old routes.

This leads to the **count-to-infinity** problem.

Q14. Difference between "good news travels fast" and "bad news travels slow" in DV.

Answer:

- **Good news travels fast** → if a new lower-cost path appears, neighbors quickly adopt it.
- **Bad news travels slow** → when a route becomes worse or fails, nodes may take many iterations to propagate the new cost (count to infinity).

Q15. What is poisoned reverse? How does it help?

Answer:
Poisoned Reverse:
If node x routes to destination y through neighbor v, then x tells v that its cost to y is ∞.

This prevents v from sending packets *back to x*, reducing loops.

Distance Vector Routing — Information Diffusion Explained

Distance Vector (DV) routing is an **iterative, distributed** routing algorithm where **routers share information with neighbors** repeatedly until all routers know the shortest paths.

This process is called:

➡ **Information Diffusion through the Network**

1️⃣ **Distance Vector State: What Does Each Node Store?**

Every router maintains a **Distance Vector (DV)**:

Destination	Cost	Next Hop

This table shows the router's *current* best-known distance to all nodes.

2️⃣ Iterative Communication (Exchange Phase)

Each node periodically sends its **entire distance vector** to all **direct neighbors**.

- Messages contain: *(Destination, Cost)* pairs
- No global knowledge
- Only local neighbor-to-neighbor communication

This is Step → "Communication"

3️⃣ Iterative Computation (Update Phase)

When a router **receives** a DV from a neighbor, it applies the **Bellman Equation**:

$$D_x(y) = \min_{v \in N(x)} \{c(x,v) + D_v(y)\}$$

Where

- $D_x(y)$: current cost at x to reach y
- $c(x,v)$: cost from x to neighbor v
- $D_v(y)$: neighbor's distance to y

This is Step → "Computation"

4️⃣ How Information Diffuses Through the Network

The algorithm works like a **wave spreading outward**, just like water ripples.

Step-by-step diffusion:

Wave 1

- Each router only knows:
    - Distance to itself = 0
    - Distance to directly connected neighbors = link cost

Wave 2

- Routers exchange their DVs
- Each router learns about nodes that are **2 hops away**

Wave 3

- Routers again exchange updated DVs
- They learn about nodes **3 hops away**

This continues **iteratively** until no more updates occur.

5️⃣ Why is it called “Diffusion”?

- Similar to **chemical diffusion**, information spreads gradually.
- No central authority.
- Each node updates based on local interactions.

After several rounds, the “distance knowledge” spreads across entire network.

6️⃣ Convergence

The process stops when:

- No router changes its distance vector after an update
- Network reaches **stable state**
- All routers know the shortest path to every destination

7️⃣ Key Characteristics of Information Diffusion in DV

✔ **Local exchange** → Only neighbors communicate
✔ **Iterative improvement** → Paths improve over time
✔ **Asynchronous** → Nodes do not need synchronized clocks
✔ **Eventually converges** → Everyone knows shortest routes
✔ **Affected by topology changes** → Diffusion starts again if a link fails

8️⃣ Summary Sentence for Notes

> “Distance Vector routing diffuses routing information through iterative neighbor-to-neighbor communication and computation. Each node repeatedly exchanges its distance vector with neighbors and updates its table using the Bellman equation. Over successive rounds, information spreads through the network until all routers converge to correct shortest paths.”

1. Quick Reminder: What Is a Distance Vector?

Each router keeps a table that says:

> **"To reach destination *D*, the minimum cost I know is *C*, and I should send packets to *next-hop N*."**

Routers **only** talk to their neighbors and exchange their tables.

2. Scenario Setup

We have three nodes:

x —— y —— z

Costs before change:

- c(x,y) = **4**
- c(y,z) = **1**
- c(z,x) = **50**

So:

- **Dy(x)** = 4 (direct)
- **Dz(y)** = 1
- **Dz(x)** = 5 (via y: 1 + 4)

Everything is normal.

PART A: GOOD NEWS → Link Cost Decreases

When c(y,x) decreases from **4** → **1**:

✔ Step t_0 — y detects decrease

- Link becomes cheaper.
- y updates Dy(x) = **1**
- Sends update to neighbors.

✔ Step t_1 — z receives update

- z now computes Dz(x) = **1 + 1 = 2** (cheaper)
- Sends update to neighbors.

✔ **Step t_2 — y receives z's update**

- y sees that its best path is still direct (**1**)
- No change → no message

Result: Only 2 iterations → converges FAST

Good news spreads fast in DV.

PART B: BAD NEWS → Link Cost INCREASES

Now the link-cost c(x,y) increases:

❗ **Cost changes from 4 → 60**

PROBLEM: DV ALGORITHM FAILS

This is where everything goes wrong.

✔ **Step t_0 — y detects increase**

Before change:

- Dy(x) = 4

After change:

Dy(x) = min(60 (direct), 1 + 5 (via z))
Dy(x) = 6

But:

❗ z's old information was wrong — it said Dz(x) = 5 (via y)

Now both routers have **stale info**.

y thinks:

"I can go through z cheaply."

z thinks:

"I can go through y cheaply."

🚨 **A LOOP IS FORMED!**

✔ **Step t_1 — y sends "new cost = 6" to z**

✔ **Step t_2 — z receives y's update**

Dz(x) = min(50 (direct), 1 + 6 (via y))
Dz(x) = 7

z sends this updated cost to y.

✔ **Step t_3 — y receives z's new cost**

y computes:

Dy(x) = 1 + 7 = 8

Sends update to z.

✔ **Step t_4 — z computes:**

Dz(x) = 1 + 8 = 9

Sends update.

🔁 **This continues:**

6 → 7 → 8 → 9 → 10 → 11 → … → 50

44 iterations!

This is the **Count-to-Infinity Problem**.

Why does count-to-infinity happen?

Because:

- Both routers **depend on each other's wrong information**
- Neither router knows the other one is routing through it
- No global knowledge exists
- DV is slow to realize a path is bad

PART C: Poisoned Reverse — The Fix (Partial Fix)

Poisoned Reverse rule:

> **If a router uses neighbor N to reach destination D, then it tells N that its cost to D is infinity.**

Meaning:

- If **z** routes to **x** via **y**,
  then **z tells y that Dz(x) = ∞**
  (a safe lie to prevent loops)

👇 **How it helps in this case:**

Before bad news:

- Dz(x) = 5 (via y)
- Therefore z **must poison reverse**:

z → y : Dz(x) = ∞

So y's table shows Dz(x) = ∞.

✔ **Now the link-cost increases: 4 → 60**

y computes:
Dy(x) = 60 (direct)

y sends Dz(x)=60 to z

z receives update:

- Now z chooses direct path:

Dz(x) = 50

- Sends Dz(x)=50 to y

y receives this update:
Dy(x) = 1 + 50 = 51

- y then poisons reverse:

$y \rightarrow z : Dy(x) = \infty$

⭐ **Result With Poisoned Reverse**

- No loop forms
- Algorithm converges quickly
- No count-to-infinity

LIMITATION

Poisoned reverse **ONLY** solves 2-node loops.

It **cannot** solve loops involving:

$A \rightarrow B \rightarrow C \rightarrow A$

Because each router only poisons **one** neighbor, not a chain of neighbors.

So count-to-infinity still happens in larger networks.

FINAL SUMMARY (Beginner-Friendly)

Concept	Explanation
DV Algorithm	Routers update costs by talking to neighbors
Good News	Spreads fast (very quick convergence)
Bad News	Spreads **very slowly**, can create routing loops
Count-to-Infinity	Costs increase step-by-step (6→7→8→…→∞)
Poisoned Reverse	Router lies to neighbor (“path is ∞”) to avoid 2-node loops
Limitation	Does NOT fix loops involving 3+ routers

LS vs DV: Full Explanation for Beginners

Routing algorithms are used by routers to find the **least-cost path** to every other router.

Two famous algorithms:

✔ **Link-State (LS)**
✔ **Distance-Vector (DV)**

They work in **very different** ways.

1. How They Share Information

DV (Distance Vector)

"**Talks only to neighbors**"

- Each router sends its **entire distance vector** (its table of distances to all destinations)
  → **only to its neighbors**.
- Over time, neighbors exchange this information repeatedly until everyone converges.

Analogy:
DV is like students whispering marks to only their bench mates.

LS (Link State)

"**Talks to everyone** (global information)"

- Each router sends **the cost of its own links**
  → **to every router in the network** (via broadcast/flooding).
- All routers receive full network info and run Dijkstra (or similar LS algorithm).

Analogy:
LS is like the teacher announcing scores on a loudspeaker to the entire class.

2. Message Complexity

LS Complexity

LS requires:

- Every router to send link info to **all** routers.
- Total messages = **O(|N| × |E|)**

Meaning:
If there are many nodes and links → LS generates **a lot of control messages**.

DV Complexity

DV requires:

- Routers to send updates **only to neighbors**
- Updates sent **every iteration**

Message complexity is **less predictable**, because:

- DV may take many iterations
- DV may react slowly to link-cost changes

DV message cost is usually **lower**, but can become large if convergence is slow.

3. Speed of Convergence

LS Speed

- LS uses **Dijkstra** → $O(N^2)$
- Converges **fast**
- No routing loops during convergence

Very stable.

DV Speed

- Can be **slow to converge**
- Can create **routing loops**
- Suffers from **count-to-infinity problem**

Much slower and less stable than LS.

4. Robustness (If a Router Misbehaves or Fails)

LS Robustness

- A router can only lie about **its own links**, nothing else.
- Each router **computes its own routing table** independently.

- Even if one router lies:
    - Damage is **limited**
    - Other routers run their own Dijkstra and do not depend on others for next steps

LS = safer in case of corruption or attack.

DV Robustness

- A DV router can lie about **its distance to ALL destinations**
- That lie is **propagated** neighbor → neighbor → whole network
- One bad router can poison entire system
  (This actually happened in 1997 — an ISP misconfiguration disconnected big parts of the Internet)

DV = more risky because wrong info spreads like a virus.

FINAL SUMMARY TABLE (Easy Exam Format)

Feature	Link State (LS)	Distance Vector (DV)
Information shared	Only own link costs	Distance vector to all destinations
Who receives updates	All routers (broadcast)	Only neighbors
Message complexity	O(N × E)	Depends, usually lower but can grow
Convergence speed	Fast (Dijkstra)	Slow, may loop
Routing loops?	No	Yes, especially after link cost increases
Count-to-infinity?	No	Yes
Robustness to misbehaving router	High	Low
Who uses it?	OSPF, IS-IS	RIP, BGP-like DV

No clear winner → **Internet uses both** depending on needs.

SHORT NOTES (Perfect for Exams / Viva)

- **LS:** Global approach, broadcasts link costs, uses Dijkstra, fast convergence, robust but requires lots of messages.
- **DV:** Local approach, exchanges distance vectors with neighbors, may loop, slow convergence, count-to-infinity problem.
- **LS safer**, **DV simpler**.
- LS used in **OSPF**, DV used in **RIP**.
- No absolute winner — each is suited for different network sizes.

QUESTIONS WITH ANSWERS (For Exams)

Q1. Why does DV suffer from count-to-infinity but LS does not?

Ans:
In DV, routers only know neighbors' distance vectors, so wrong information can circulate and keep increasing.
In LS, routers have a full network map and run Dijkstra, so they detect link failures immediately.

Q2. Which algorithm has higher message complexity and why?

Ans:
LS has higher message complexity because every router must broadcast its link-state information to all routers, requiring O(N × E) messages.

Q3. Why is LS considered more robust than DV?

Ans:
In LS, each router computes routes independently and only advertises its own link costs. In DV, a router can lie about all destinations and the lie spreads through the network.

Q4. Which algorithm converges faster and why?

Ans:
LS converges faster because Dijkstra computes shortest paths immediately after receiving full link-state information.
DV waits for iterative updates from neighbors and may form loops.

Q5. Name one protocol that uses LS and one that uses DV.

Ans:

- LS → OSPF
- DV → RIP

OSPF – Exam-Focused Notes

1. Why Intra-AS Routing?

Routers in the Internet are divided into **Autonomous Systems (ASs)** because:

(a) Scale

- The Internet has **hundreds of millions of routers**.
- Storing global routing info in every router = **huge memory, huge control traffic**.
- Distance-Vector (DV) algorithm **won't converge** for such a large network.

➡ ASs reduce complexity by dividing routers into manageable groups.

(b) Administrative Autonomy

- Each ISP wants **full control** over its internal network.
- ISPs may hide internal network structure.
- They can choose **their own internal routing protocol**.

➡ Each AS runs its own protocol → called **Intra-AS Routing Protocol**.

2. What is OSPF?

OSPF = Open Shortest Path First

- Used inside an AS (intra-AS).
- Publicly available (unlike old Cisco EIGRP).
- OSPFv2 defined in **RFC 2328**.
- It is a **Link State (LS)** routing protocol.

3. How OSPF Works (Core Concepts)

(1) Link-State Flooding

- Every router sends **Link-State Advertisements (LSAs)** to **all routers in the AS**.
- Triggered by:

    - Link cost change
    - Link up/down
    - Periodic refresh (every 30 minutes) → for robustness

(2) Each Router Builds a Full Map

- Every router builds a complete **graph** of the AS.
- Runs **Dijkstra's Algorithm** locally → shortest path tree.

(3) Link Costs

- Set by **network administrator**.
- Not mandated by OSPF.
- Options:
    - All cost = 1 → **minimum hop routing**
    - Inversely proportional to capacity → avoids slow links

4. Extra Features of OSPF (Exam Important)

(a) Security

- Supports **authentication**:
    - Simple password (plaintext)
    - **MD5 authentication** → secure, with sequence numbers to prevent replay attacks

(b) Multiple Equal-Cost Paths

- OSPF supports **load balancing** across multiple equal-cost routes.

(c) Multicast Support

- MOSPF (Multicast OSPF) extends OSPF for multicast routing.

(d) Hierarchy Support

OSPF supports **multi-area hierarchical design**:

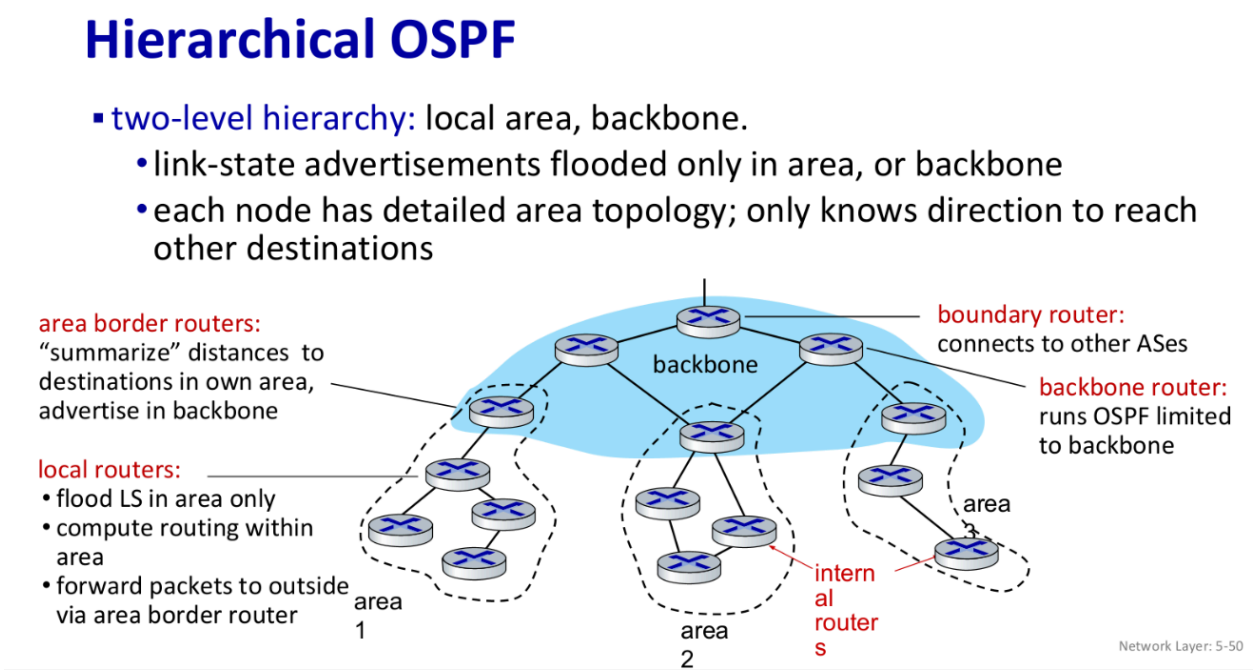
1. **Backbone area (Area 0)**
    - Mandatory
    - Connects all other areas

- Routes inter-area traffic

2. **Regular areas**
   - Run their own LS algorithm
   - Routers within an area flood LSAs only inside the area

3. **Area Border Routers (ABRs)**
   - Connect an area to the backbone
   - Summarize and distribute routing info

➡ Routing path:
Intra-area → ABR (Area Border Router) → backbone → ABR (Area Border Router) → destination area



<u>Super Short Summary (Excellent for last-minute revision)</u>

- OSPF = **Link-state**, uses **Dijkstra**, sends **LSAs**, builds **full topology**, and supports **multiple areas**.
- Provides **authentication**, **equal-cost multipath**, and **multicast support**.
- Reduces complexity & gives ISPs **autonomy**.

Most Likely Exam Questions (with Answers)

1. Why do we need Autonomous Systems (ASs) in the Internet?

Ans:
(1) **Scale:** Too many routers → global routing becomes impossible.
(2) **Administrative Autonomy:** ISPs want to control their own internal routing.

2. What is OSPF? State its main characteristics.

Ans:
OSPF is an **intra-AS link-state routing protocol** that uses **flooding** and **Dijkstra's algorithm**.
Features include:

- Authentication (simple + MD5)
- Equal-cost multipath
- Multicast support (MOSPF)
- Hierarchical areas (backbone + ABRs)

3. How does OSPF differ from Distance-Vector (DV) routing?

Ans:

- OSPF is **link state**; DV is **distance vector**.
- OSPF floods LSAs to **all routers**; DV sends info **only to neighbors**.
- OSPF builds full topology + runs Dijkstra; DV uses Bellman-Ford.
- OSPF converges faster and is more scalable.

4. Explain OSPF link-state advertisements (LSAs).

Ans:
LSAs carry info about router links (cost, up/down).
Sent:

- When link changes
- Periodically every 30 minutes (for robustness)
  Flooded to **all routers in AS**.

5. What is the role of OSPF hierarchical areas?

Ans:
Reduce LS overhead and complexity by dividing AS into:

- **Area 0 (Backbone)** → carries inter-area traffic
- **Other areas** → manage their own routing
- **ABRs** → connect areas

6. What are different OSPF authentication methods?

Ans:

1. **Simple authentication:** password in plaintext
2. **MD5 authentication:** secure hash + secret key + sequence number

7. Why is link cost important in OSPF?

Ans:
OSPF uses link cost to compute shortest paths.
Admin may set cost = 1 (min-hop) or inversely proportional to capacity (traffic engineering).

8. What is the purpose of the OSPF backbone (Area 0)?

Ans:
Ensures **connectivity between all other areas** → handles inter-area routing.

9. Define Area Border Router (ABR).

Ans:
A router that:

- Connects an area to Area 0
- Summarizes routing info.
- Participates in both backbone and local area routing

1. What is BGP?

- **Inter-AS (Inter-Autonomous System) routing protocol** used by all ASes on the Internet.
- Runs **over TCP port 179**.
- **Decentralized, asynchronous distance-vector–like protocol.**
- Most important protocol for **gluing ISPs together**.

2. Why is BGP needed? (Role of BGP)

BGP is responsible for:

a. Prefix Reachability

- Routers do NOT advertise individual IPs; they advertise **CIDR prefixes**
  e.g., 138.16.68.0/22 → represents **1024 IPs**.
- BGP allows subnets to tell the world:
  "I exist and I am reachable here!"

b. Best Route Selection

- Routers may learn **multiple paths** to a prefix.
- BGP decides the **best path** based on:
  - **Policy** (business relationships)
  - **AS-path length**
  - Other BGP attributes.

BGP PROTOCOL MESSAGES (VERY IMPORTANT FOR EXAMS)

1. OPEN Message

- First message sent after establishing a TCP connection.
- Purpose:
  - Establish BGP session
  - Authenticate peer
- If an OPEN is received correctly, peer responds with **KEEPALIVE**.

2. UPDATE Message

- Most important BGP message.
- Used to:
  - **Advertise new paths**
  - **Withdraw old/invalid paths**

Contains:

- Prefix
- AS-Path
- Next-hop information

3. KEEPALIVE Message

- Used when no UPDATES are being sent.

- Ensures session is still up.
- Also acts as acknowledgment for an OPEN message.

4. NOTIFICATION Message

- Sent when an error occurs in a previous BGP message.
- Used to **terminate the connection**.
- Example: malformed UPDATE → send NOTIFICATION.

3. BGP Routing Table Entry

A forwarding table entry looks like:

(prefix, outgoing interface)

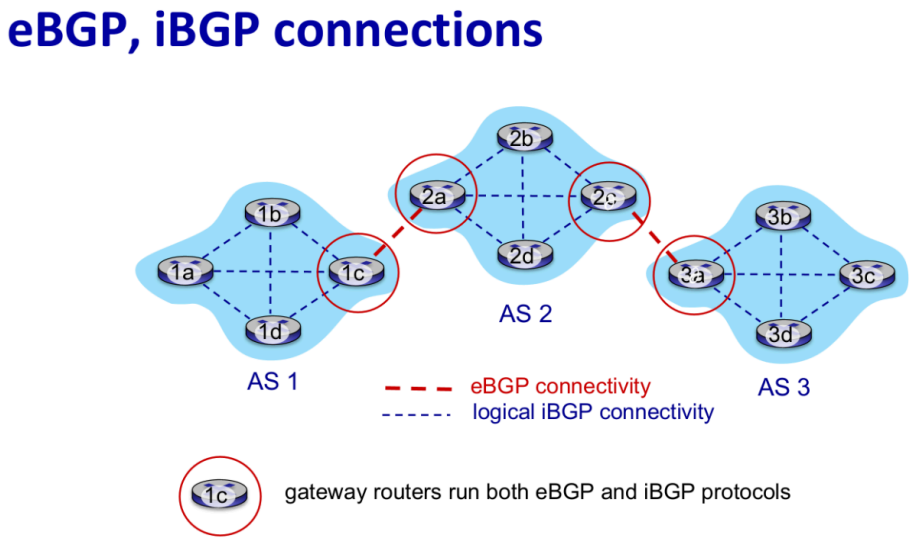
4. Types of BGP Connections

a. eBGP (External BGP)

- Connects **gateway routers of different ASes**.
- Used to **learn reachability from other ASes**.

b. iBGP (Internal BGP)

- Connects **routers inside the same AS** (often full mesh).
- Used to **propagate BGP-learned routes within AS**.
- Doesn't correspond to physical links — runs over TCP.



5. How BGP Advertises Reachability

Example: Prefix **x** is in **AS3** → must be advertised to AS2 and then AS1.

Step-by-step Propagation

1. **AS3's router 3a → 2c**
   Advertisement: "AS3 x" **(eBGP)**
2. **2c → all routers in AS2**
   "AS3 x" via **iBGP**
3. **AS2 gateway 2a → AS1's 1c**
   Advertisement: "AS2 AS3 x" **(eBGP)**
4. **1c → all routers in AS1**
   "AS2 AS3 x" via **iBGP**

Now all routers in AS1 and AS2 know:

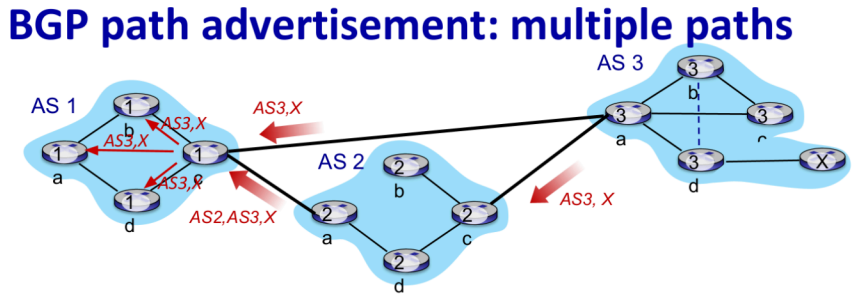
- Prefix **x** exists
- AS path to reach x

6. Multiple Paths Example

If another link exists (e.g., 1d → 3d), AS1 may learn two paths:

1. **Path 1 (through AS2):**
   "AS2 AS3 x"
2. **Path 2 (direct to AS3):**
   "AS3 x"

BGP will choose best path using its **path-selection rules**.



gateway router may learn about multiple paths to destination:

- AS1 gateway router 1c learns path *AS2,AS3,X* from 2a
- AS1 gateway router 1c learns path *AS3,X* from 3a
- based on *policy,* AS1 gateway router 1c chooses path *AS3,X* and advertises path within AS1 via iBGP

BGP Route Selection & IP-Anycast — Notes (Easy & Complete)

1. BGP Attributes

When a router advertises a prefix, the advertisement contains *attributes*.
A prefix + attributes = **BGP Route**.

Important Attributes

1. **AS-PATH**
    - List of ASes that the route advertisement has passed through.
    - Used to:
        - Detect routing loops (if router sees its own AS → reject)
        - Select shortest AS-path route

2. **NEXT-HOP**
    - IP address of the router **starting the AS-PATH**.
    - Often from a neighboring AS.
    - Used with **intra-AS routing (OSPF/IS-IS)** to find shortest internal path to that border router.

2. Two Types of BGP Sessions

eBGP

- Between routers in **different ASes**
- Used to **learn routes from external networks**

iBGP

- Between routers **inside the same AS**
- Used to **propagate eBGP-learned routes** to all routers within AS

3. Hot Potato Routing (Key Concept!)

Goal: **Send packets out of the AS as fast as possible**, i.e., to the **closest NEXT-HOP** gateway.

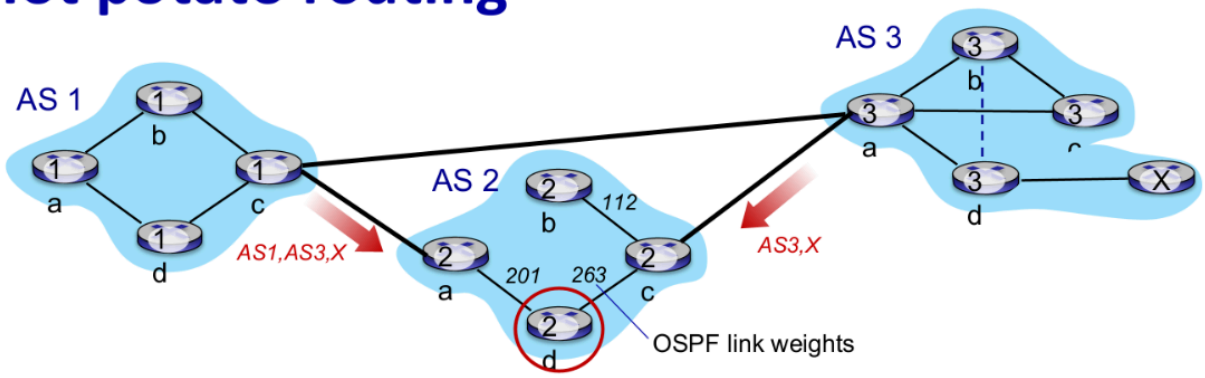
Steps:

1. Learn that prefix X is reachable via multiple BGP routes
2. Check intra-AS routing table → find cost to each NEXT-HOP

3. Choose path with **least cost to NEXT-HOP**
4. Update forwarding table: (prefix x, interface I)

This is **selfish routing**:
It prefers minimizing **internal cost** only, ignoring external hops.

Hot potato routing



- 2d learns (via iBGP) it can route to X via 2a or 2c
- hot potato routing: choose local gateway that has least *intra-domain* cost (e.g., 2d chooses 2a, even though more AS hops to *X*): don't worry about inter-domain cost!

4. Full BGP Route Selection Algorithm (Important!)

If multiple routes exist → apply these rules **in order**:

1 Highest Local Preference

- Local pref = administrative policy inside AS
- Highest value wins
- Overrides all others

2 Shortest AS-PATH

- Fewer AS hops wins
- BGP is a Path-Vector protocol → shorter path = preferred

3 Hot Potato Routing

- If tied above → choose route with **closest NEXT-HOP**

4️⃣ Lowest Router ID

- Tie-breaker (least important)

1. A route is assigned a **local preference** value as one of its attributes (in addition to the AS-PATH and NEXT-HOP attributes). The local preference of a route could have been set by the router or could have been learned from another router in the same AS. The value of the local preference attribute is a policy decision that is left entirely up to the AS's network administrator. (We will shortly discuss BGP policy issues in some detail.) The routes with the highest local preference values are selected.
2. From the remaining routes (all with the same highest local preference value), the route with the shortest AS-PATH is selected. If this rule were the only rule for route selection, then BGP would be using a DV algorithm for path determination, where the distance metric uses the number of AS hops rather than the number of router hops.
3. From the remaining routes (all with the same highest local preference value and the same AS-PATH length), hot potato routing is used, that is, the route with the closest NEXT-HOP router is selected.
4. If more than one route still remains, the router uses BGP identifiers to select the route; see [Stewart 1999].

5. IP-Anycast

Purpose: Send clients to the **nearest server** (by BGP decision), even though multiple servers share the **same IP address**.

How it works:

1. Many servers around the world are configured with **same IP**
2. Each server advertises that IP via BGP
3. Routers treat these as different *paths to the same destination*
4. BGP chooses best path (shortest AS-PATH, local pref, hot potato)
5. User request goes to *nearest* server

Where IP-use Anycast?

✔ **DNS Root Servers (extensively used!)**
✔ DDoS protection
✖ **CDNs rarely use it** for TCP because packets might reach different servers

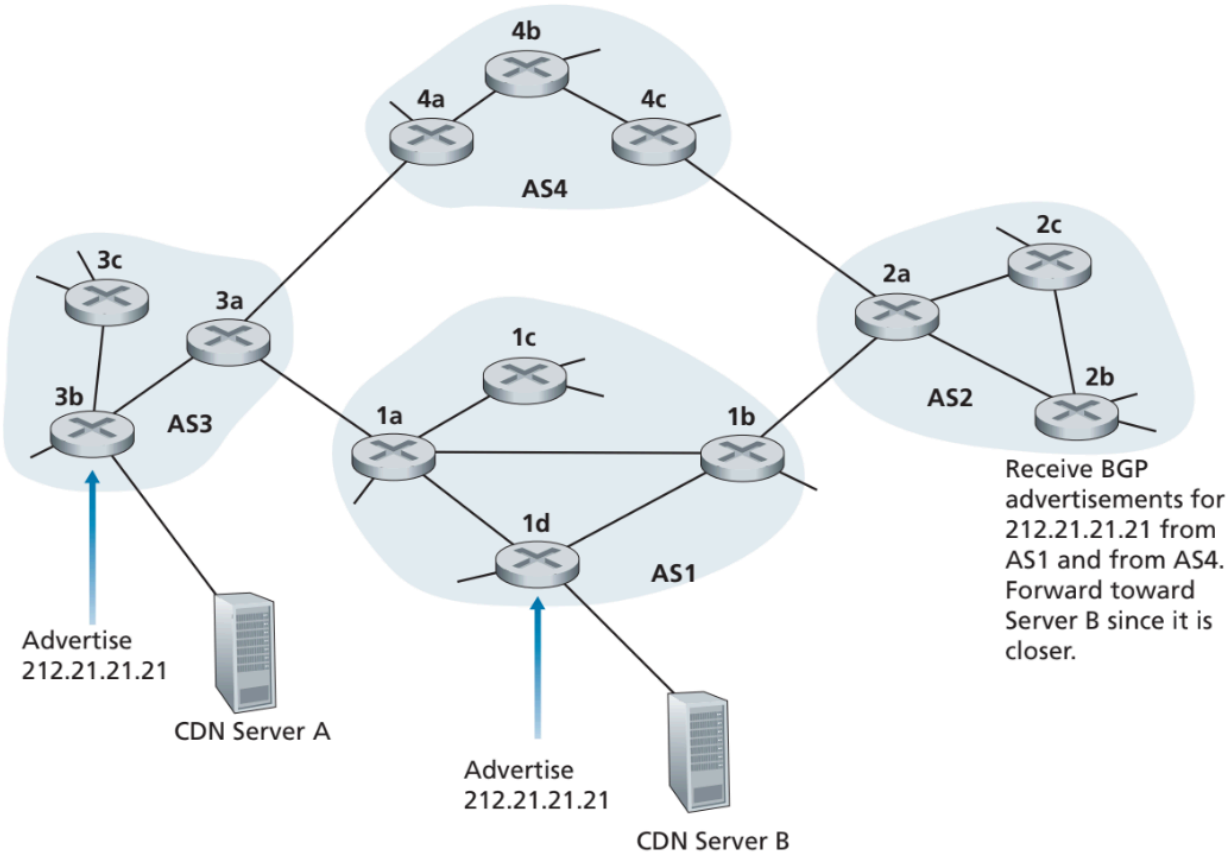


Figure 5.12 ♦ Using IP-anycast to bring users to the closest CDN server

BGP: Achieving Policy via Advertisements — Easy Notes

BGP is not only a routing protocol — **it is a policy enforcement tool**.

ISPs use BGP **advertisements** to control **which routes they want to carry** and **which routes they want to hide**.

1. Key Idea: ISPs block/allow advertisements based on their routing policy

BGP allows a network to **choose which routes to advertise to neighbors**.
This gives control over:

- Who can send traffic through your AS
- Who cannot
- Preventing free transit traffic
- Following business relationships (customer–provider)

2. Customer–Provider Relationship Rules

In real-world Internet routing:

- **Customers PAY providers** → Providers carry **all** traffic to/from customer.
- **Providers DO NOT PAY customers** → Customers do **not** carry traffic between providers.

Meaning:

> ISPs do **not** want to carry transit traffic between other ISPs because they earn **no revenue**.

3. Example 1: Why B does NOT advertise route to C

Diagram scenario:

- A, B, C = provider networks
- w = customer of A
- A advertises route **A–w** to B
- B learns path **B–A–w**
- C is another provider

Why does B NOT advertise B–A–w to C?

Because:

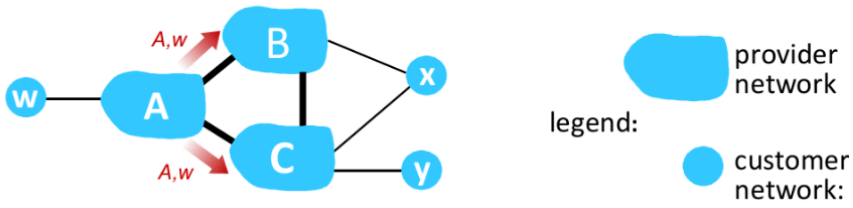
- C → B → A → w would be **transit traffic**
- None of A, C, or w are customers of B
- So **B earns nothing** for carrying that traffic

Therefore:

B refuses to advertise the route to C
→ C does **not** learn path C–B–A–w
→ C must route using its own path (C → A → w)

This is a **classic real-world policy**.

BGP: achieving policy via advertisements



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical "real world" policy)

- A advertises path Aw to B and to C
- B *chooses not to advertise* BAw to C!
  - B gets no "revenue" for routing CBAw, since none of C, A, w are B's customers
  - C does *not* learn about CBAw path
- C will route CAw (not using B) to get to w

4. Resulting Behavior

- BGP routing becomes **policy-based**, not just shortest-path.
- ISPs hide certain routes to **avoid becoming free transit**.

5. Example 2: Dual-Homed Customer x (Very Important!)**

Setup:

- x is a customer of two ISPs (dual-homed)
  - Connected to A and B
- x is also a customer of C

Policy goal:

x does not want to route from B to C through itself
(because x does not want to help B reach C for free)

How x enforces this:

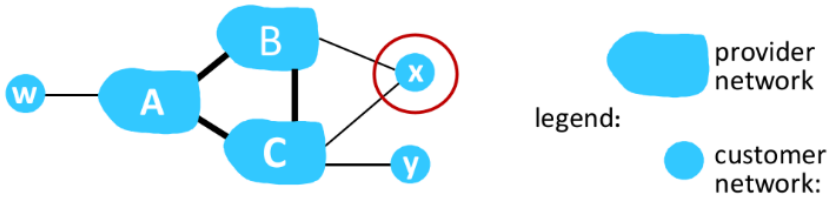
x simply does NOT advertise a route to C when speaking to B

Therefore:

- B does NOT learn route B → x → C
- B will NOT use x as a transit to reach C.
- Transit is blocked.

This is how customers prevent being used as a **free transit ISP**.

BGP: achieving policy via advertisements (more)



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical "real world" policy)

- A,B,C are provider networks
- x,w,y are customer (of provider networks)
- x is dual-homed: attached to two networks
- *policy to enforce:* x does not want to route from B to C via x
  - .. so x will not advertise to B a route to C

6. Summary of BGP Policy via Advertisements

Entity	Advertise to whom?	Why?
Provider → advertised to everyone	Customers, peers, upstream	They are paid

Customer → advertise to providers	Providers	They want reachability
Customer → NOT advertise peer/provider routes	Do not advertise routes learned from other providers	To avoid being used as free transit
ISPs block routes that give no revenue	Example: B does not advertise B–A–w to C	Avoid transit traffic

7. One-Sentence Takeaway

> **BGP implements economic and business policies by controlling which routes are advertised to which neighbors.**

Software Defined Networking (SDN) – NOTES

1. Traditional Per-Router Control Plane

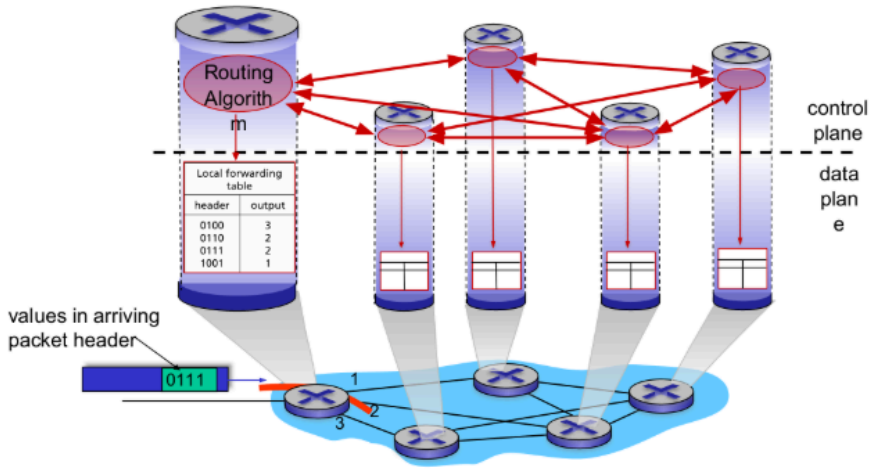
- In traditional networks:
  - Each router contains its own **routing algorithm**.
  - Routers exchange information with neighbors (via DV or LS protocols).
  - Each router independently computes its **forwarding table**.
- Control plane and data plane are **in the same device**.

Diagram meaning:

- **Data plane**: where packets are forwarded based on the forwarding table.
- **Control plane**: routing algorithm running inside each router.

Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane to compute forwarding tables

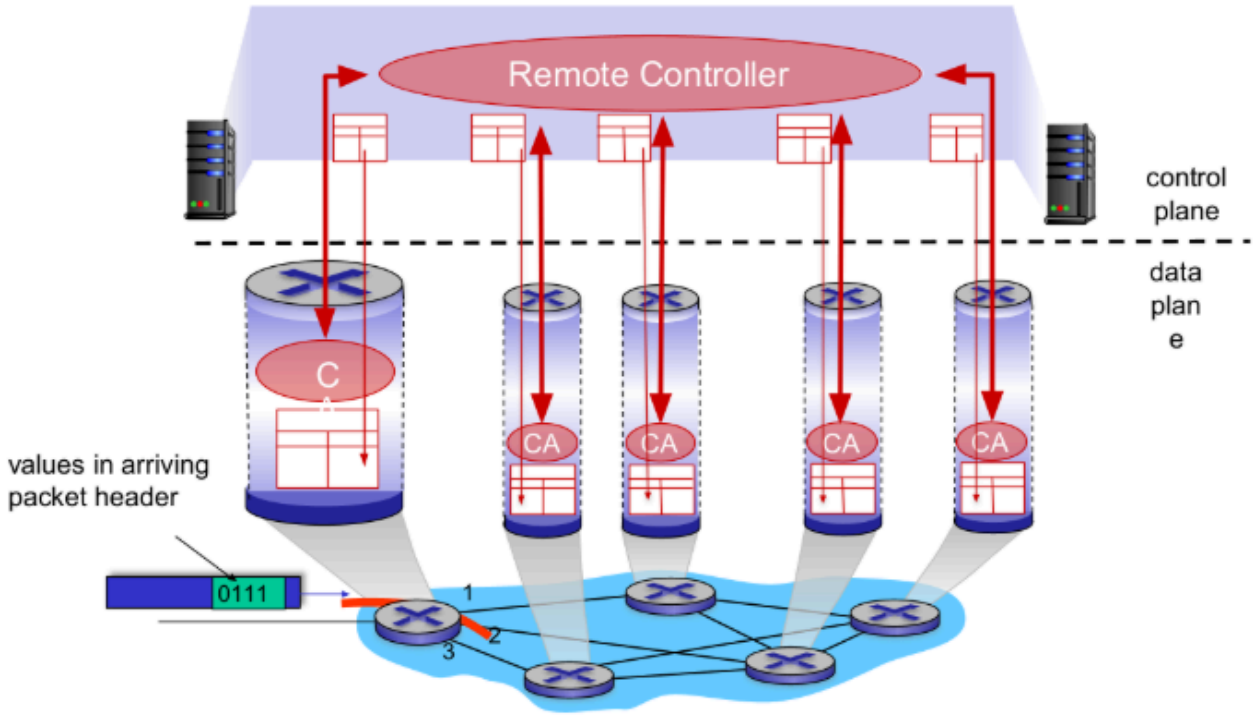


2. SDN Control Plane

- SDN moves the **control plane out of the routers**.
- A **remote, logically centralized controller** computes forwarding rules for all routers/switches.
- Switches only perform **simple matching + forwarding** (flow tables).

Advantages:

- Central "brain" → global network view.
- Remote controller installs forwarding tables in all switches.



3. Why Logically Centralized Control Plane?

(a) Easier Network Management

- Avoids router-by-router manual configuration.
- Single point for updating policies → fewer misconfigurations.

(b) Greater Flexibility

- Table-based forwarding (like OpenFlow) lets operators define:
  - Any match conditions (IP, MAC, TCP, protocol)
  - Actions (forward, drop, mirror)
- Can **program** routers (traffic engineering, load balancing, ACLs, NAT rules).

(c) Centralized Programming

- Controller computes tables centrally.
- Easy to implement flexible routing logic.

(d) Distributed Programming is Hard

- Traditional networks compute routes via distributed algorithms.
- Hard to implement new policies or customized routing.

(e) Open (Non-Proprietary) Control Plane

- Uses open interfaces (OpenFlow, gRPC, REST APIs).
- NOT dependent on vendor-specific firmware.

(f) Enables Rapid Innovation

- Developers can write control apps without modifying routers.
- “Let 1000 flowers bloom” — anyone can innovate on top of SDN.

Traffic Engineering is Hard in Traditional Routing

Consider a network operator wanting specific routing behavior.

Case 1: Force traffic u → z to follow uvwz instead of uxyz

- Traditional routing only allows **link weight changes** as control mechanism.
- Changing weights affects **all** flows, not just one.
- No fine-grained control.

Case 2: Split u → z traffic on two different paths (load balancing)

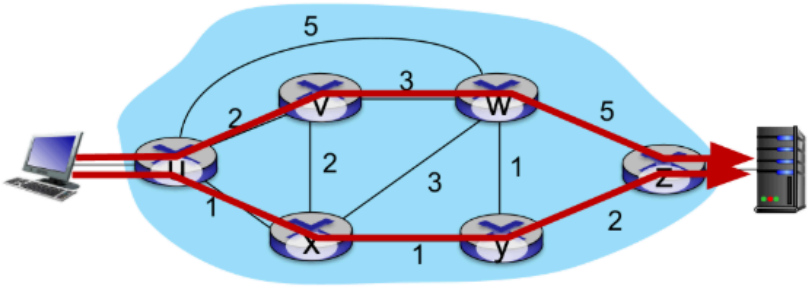
- Not possible with standard LS/DV routing.
- Requires special protocols or hacks.

Case 3: w wants to route Red and Blue traffic differently to z

- Traditional routing = **destination-based forwarding only**.
- Cannot differentiate traffic by:
    - Application
    - Source
    - TCP/UDP port
    - Packet type

➡ With destination-only routing, *fine-grained traffic differentiation is impossible.*

Traffic engineering: difficult with traditional routing

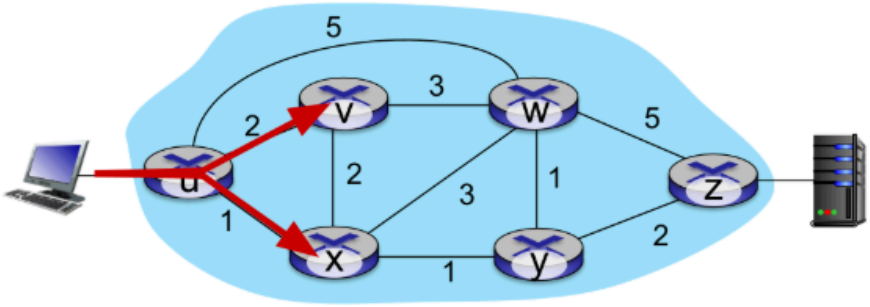


Q: what if network operator wants u-to-z traffic to flow along *uvwz*, rather than *uxyz*?

A: need to re-define link weights so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

link weights are only control "knobs": not much control!

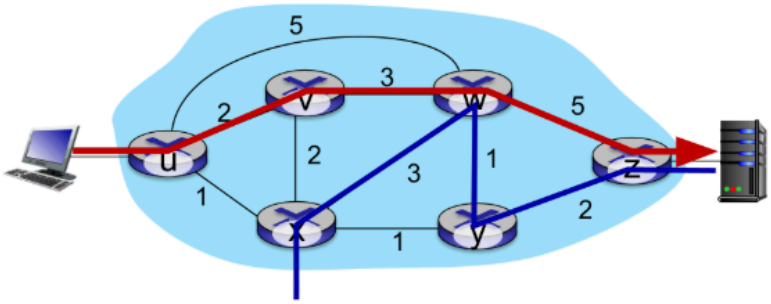
Traffic engineering: difficult with traditional routing



Q: what if network operator wants to split u-to-z traffic along uvwz *and* uxyz (load balancing)?

A: can't do it (or need a new routing algorithm)

Traffic engineering: difficult with traditional routing



Q: what if w wants to route blue and red traffic differently from w to z?

A: can't do it (with destination-based forwarding, and LS, DV routing)

We learned in Chapter 4 that generalized forwarding and SDN can be used to achieve *any* routing desired

Summary of Why SDN Is Powerful

- **Centralized logic** → global optimization.
- **Programmable forwarding** → flexible traffic control.
- **Open interfaces** → fast innovation.
- **Fine-grained match conditions** → per-flow, per-app, per-user routing.
- Solves limitations of traditional LS/DV routing.

Software-Defined Networking (SDN) – Four Key Characteristics (Detailed Notes)

SDN (Software-Defined Networking) fundamentally changes how networks are designed and controlled. According to **Kreutz (2015)**, SDN has **four core characteristics**:

1. Flow-Based Forwarding

Traditional Networks

- Routers forward packets **only based on destination IP address**.

- Fixed, limited flexibility.

SDN Approach

- SDN switches use **flow tables**.
- A “flow” can be defined using **any combination of packet header fields**, not just destination IP.
- OpenFlow 1.0 supports matching on **11 header fields**, including:
    - MAC addresses
    - VLAN ID
    - IP src/dst
    - Transport-layer ports
    - Protocol type, etc.

Key Advantage

- **Fine-grained control** over traffic.
- Operators can route or filter packets based on:
    - Application type
    - User identity
    - TCP port (e.g., load balancing)
    - Security policies
- Forwarding behavior becomes **programmable and flexible**, not fixed.

2. Separation of Data Plane and Control Plane

Traditional Networks

- Both planes exist **inside the same router**.
    - Router runs routing algorithms (control plane).
    - Router forwards packets (data plane).

SDN Approach

- **Data Plane:** Simple, fast switches that only **match + action** rules from flow tables.
- **Control Plane:** External software that programs these switches.

Why this separation matters

- Simplifies switch hardware → faster, cheaper switches.

- Makes the control logic:
    - Easily upgradable
    - More flexible
    - Centralized

Result

- Enables **centralized network-wide decision making** rather than per-router decisions.

3. Network Control Functions Are External to Switches

Where does the control logic live?

- In the **SDN controller** (also called the network operating system).
- Controller is **logically centralized**, but physically may run on multiple servers for:
    - Scalability
    - Reliability
    - High availability

Responsibilities of the Controller

1. Maintains **global network state**
    - Link status
    - Switch status
    - Host positions
2. Provides this information to **network control applications** (routing, load balancing, security).
3. Offers APIs to:
    - Monitor switches
    - Install flow rules
    - Modify forwarding behavior

Controller + Apps = SDN Control Plane

- **Controller:** Operating system of the network
- **Applications:** The “brains” using APIs to control the network

4. A Programmable Network

Programming via Control Applications

- Developers can write applications that:
    - Compute paths (routing)
    - Enforce security policies (ACLs)
    - Balance load across servers
    - Prioritize QoS traffic (voice/video)

- These apps interact with the controller using **open APIs** like:
    - Northbound APIs (controller ⇆ apps)
    - Southbound APIs (controller ⇆ switches) such as OpenFlow

Why this is powerful

- Network behavior becomes **software-driven**.
- Changes can be applied dynamically:
    - Congestion control
    - Traffic engineering
    - Failure recovery
    - Mobility support

Unbundling of the Network

SDN breaks the traditional router model into three independent components:

1. **Switch hardware** (data plane)
2. **Network OS / Controller**
3. **Control applications**

Each can be from different vendors → increases innovation and reduces vendor lock-in.

Summary Table

Characteristic	Meaning	Key Benefits
Flow-based Forwarding	Forward using any header field, not only destination IP	Fine-grained control, flexible policies
Separation of Data and Control Plane	Switch forwards; controller decides	Simpler switches, easier management
External Network Control	Control logic runs on remote controller	Global network view, central decisions

Programmable Network	Control apps program network behavior	Rapid innovation, dynamic policies, open ecosystem

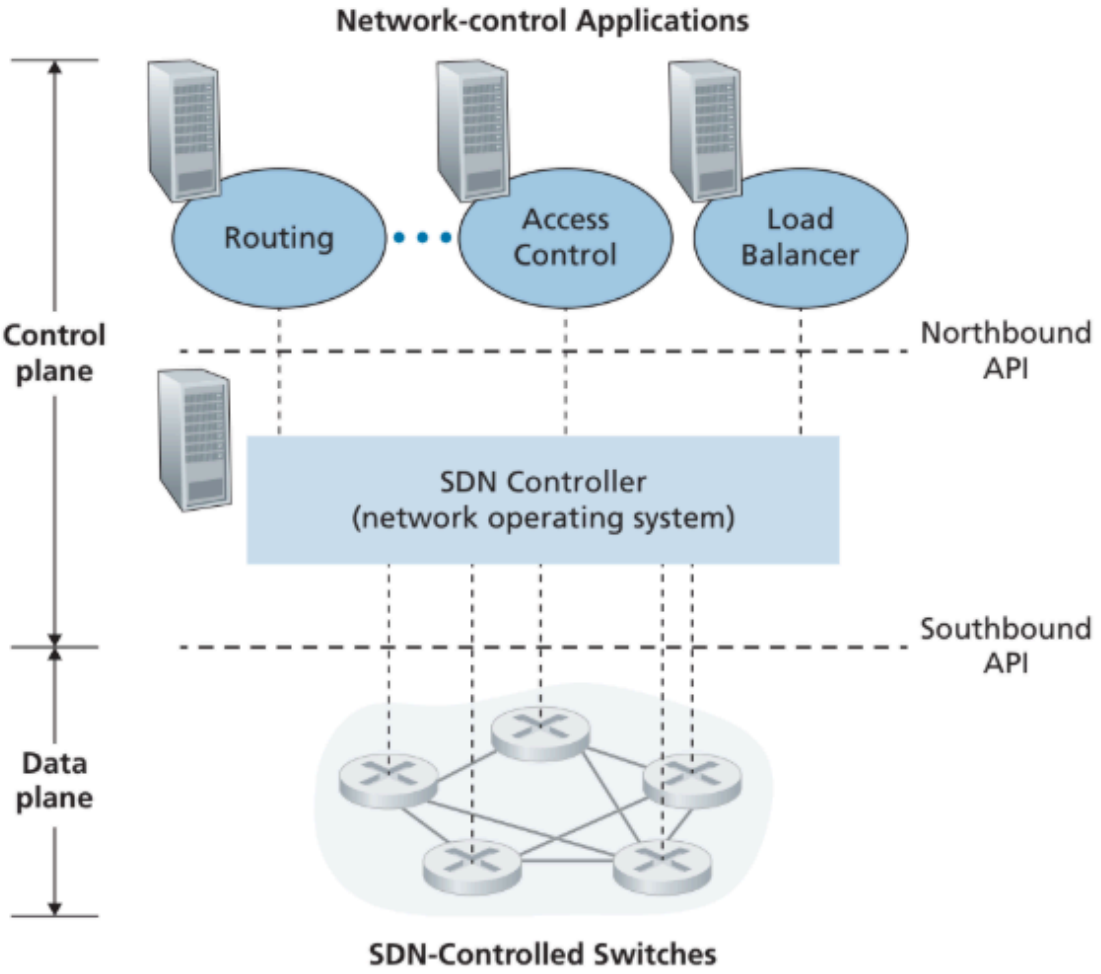


Figure 5.14 ♦ Components of the SDN architecture: SDN-controlled switches, the SDN controller, network-control applications

Components of an SDN Controller (Brief)

1. Communication Layer (Southbound Interface)

- Communicates with switches using protocols like **OpenFlow**.
- Installs flow table entries and collects switch/traffic information.

2. Network-Wide State Management

- Maintains a **global view** of the entire network (links, switches, hosts, stats).
- Acts as a **distributed database** to store and update network state.

3. Interface to Network-Control Applications (Northbound Interface)

- Provides APIs (e.g., **REST API**) and abstractions for apps.
- Allows applications like routing, access control, and load balancing to control the network.

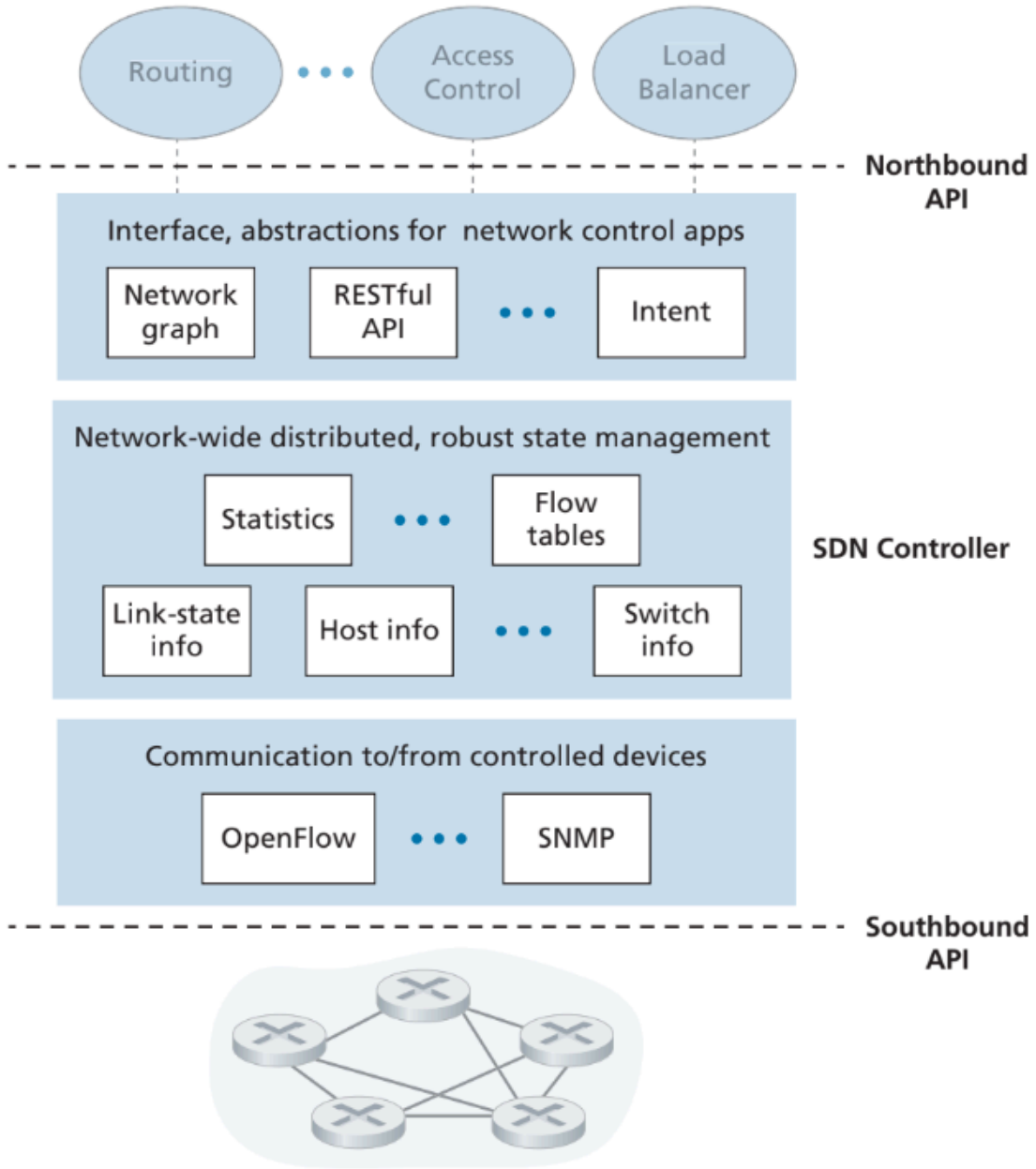


Figure 5.15 ♦ Components of an SDN controller

OpenFlow Protocol — Brief Notes

Overview

- Runs **between SDN controller and switches**.
- Uses **TCP** for message exchange (with optional encryption).
- Defines how the controller **manages switch flow tables**.
- Different from the **OpenFlow API** (used by apps to specify forwarding rules).

Three Classes of OpenFlow Messages

1. Controller-to-Switch Messages

Used by controller to manage switches.

- **features** → controller asks switch capabilities; switch replies
- **configure** → get/set switch configuration
- **modify-state** → add/delete/modify flow table entries
- **packet-out** → instruct switch to send a specific packet out a port

2. Switch-to-Controller (Asynchronous) Messages

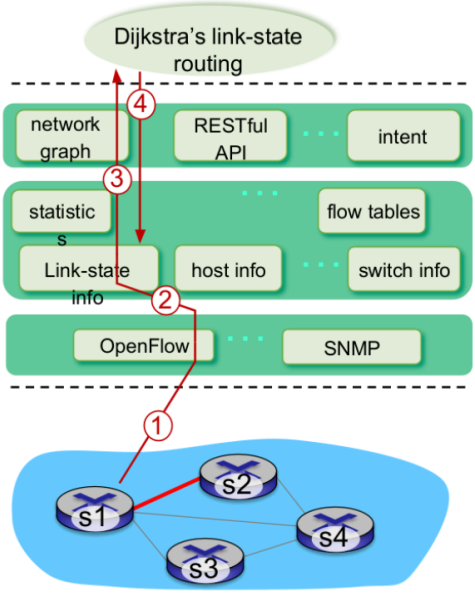
Sent from switch when events occur.

- **packet-in** → switch sends a packet to controller (no matching flow entry)
- **flow-removed** → tells controller a flow entry was deleted
- **port-status** → informs controller of port changes (up/down/etc.)

3. Symmetric Messages

- Miscellaneous messages sent either way (e.g., hello messages).

SDN: control/data plane interaction example



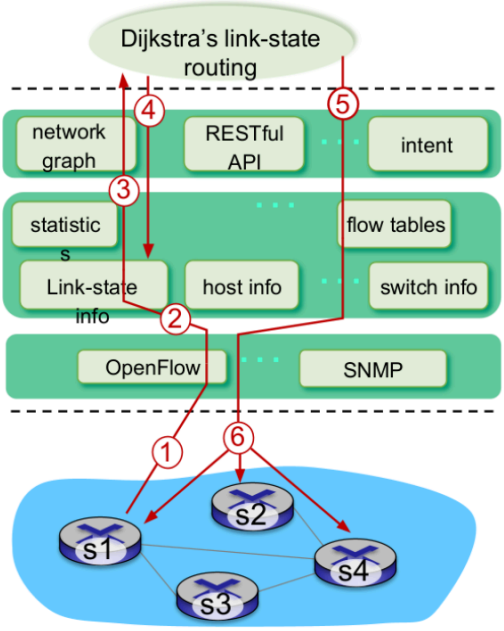
① S1, experiencing link failure uses OpenFlow port status message to notify controller

② SDN controller receives OpenFlow message, updates link status info

③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.

④ Dijkstra's routing algorithm access network graph info, link state info in controller, computes new routes

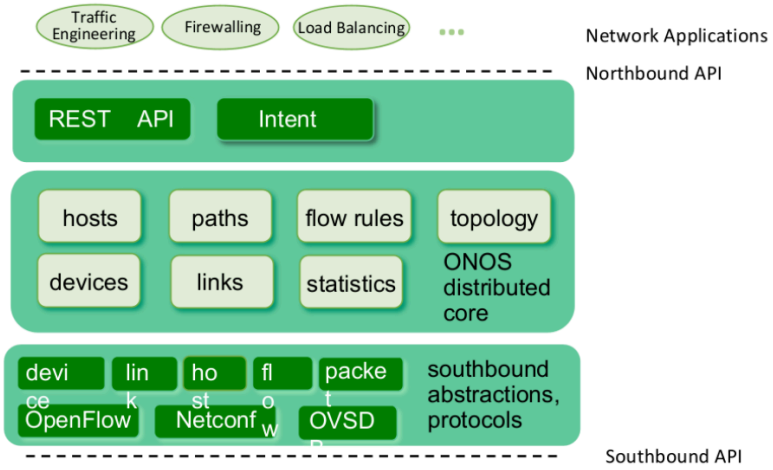
SDN: control/data plane interaction example



⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed

⑥ controller uses OpenFlow to install new tables in switches that need updating

ONOS SDN controller



- control apps separate from controller
- intent framework: high-level specification of service: what rather than how
- considerable emphasis on distributed core: service reliability, replication performance scaling

ONOS: early, influential open-source SDN controller

P12. Describe how loops in paths can be detected in BGP.

P13. Will a BGP router always choose the loop-free route with the shortest ASpath length? Justify your answer.

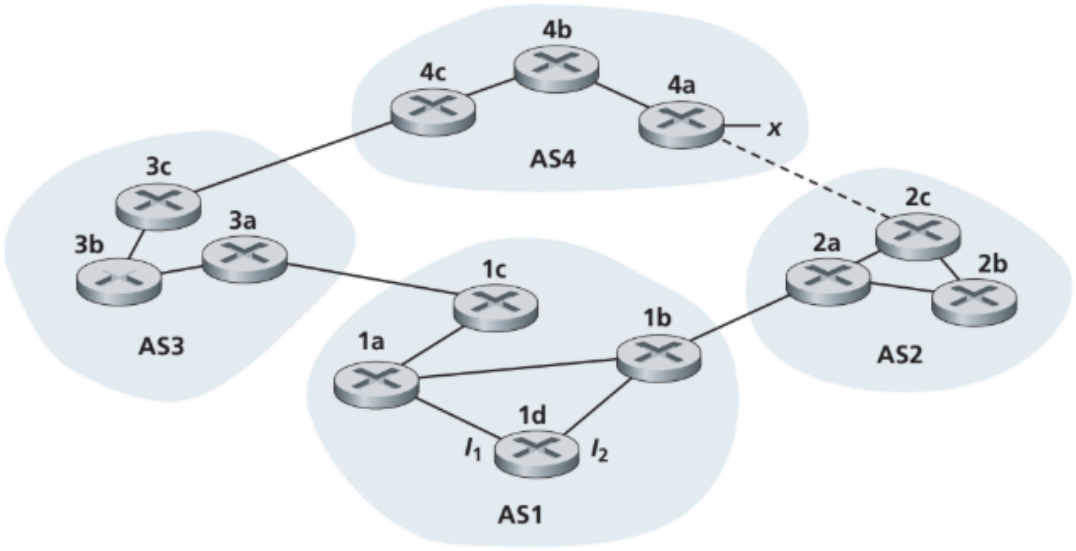
P14. Consider the network shown below. Suppose AS3 and AS2 are running OSPF for their intra-AS routing protocol. Suppose AS1 and AS4 are running RIP for their intra-AS routing protocol. Suppose eBGP and iBGP are used for the inter-AS routing protocol. Initially suppose there is *no* physical link between AS2 and AS4.

a. Router 3c learns about prefix *x* from which routing protocol: OSPF, RIP, eBGP, or iBGP?

b. Router 3a learns about *x* from which routing protocol?

c. Router 1c learns about *x* from which routing protocol?

d. Router 1d learns about *x* from which routing protocol?



P12. Describe how loops in paths can be detected in BGP.

BGP (Border Gateway Protocol) detects loops using the **AS-PATH** attribute.

- **Mechanism:** When a BGP router advertises a route to an eBGP peer (a peer in a different Autonomous System), it prepends its own AS number to the AS-PATH list.
- **Detection:** When a BGP router receives a route advertisement, it scans the AS-PATH attribute. If the router sees its **own AS number** already present in the AS-PATH list, it knows that the routing update has already passed through its autonomous system.

- **Action:** To prevent a routing loop, the router immediately rejects/discards that route advertisement.

P13. Will a BGP router always choose the loop-free route with the shortest AS path length? Justify your answer.

No, a BGP router will not always choose the route with the shortest AS path length.

Justification:

BGP uses a specific sequence of criteria to select the best path. While AS-PATH length is an important metric, it is not the top priority. The Local Preference attribute is evaluated before the AS-PATH length.

1. **Local Preference:** This is the highest priority attribute. If a route has a longer AS-PATH but a higher Local Preference (which is often set by network administrators to enforce policy, such as preferring a cheaper link), the router will choose that route over a shorter one with lower preference.
2. **AS-PATH:** The shortest AS-PATH is only chosen if the Local Preference values are equal.

Therefore, policy decisions (reflected in Local Preference) can override path efficiency (AS path length).

P14. Routing Protocols in the Network Diagram

In this scenario, prefix **x** originates in **AS4**. Since there is no link between AS2 and AS4, the route must propagate from **AS4 $\rightarrow$ AS3 $\rightarrow$ AS1**. The usage of eBGP vs. iBGP depends on whether the communicating routers are in the same AS or different ones.

a. Router 3c learns about prefix x from which routing protocol?

Answer: eBGP

- **Reasoning:** Router 3c (in AS3) is connected to Router 4c (in AS4). Since they are in different Autonomous Systems, they exchange routing information using **eBGP** (Exterior BGP).

b. Router 3a learns about x from which routing protocol?

Answer: iBGP

- **Reasoning:** Router 3a learns the route from Router 3c. Both routers are inside the same Autonomous System (AS3). The protocol used to disseminate external routing information *within* an AS is **iBGP** (Interior BGP).
    - *Note:* While OSPF is running in AS3, it is typically used to handle internal routing (finding the path from 3a to 3c), whereas the external prefix x itself is carried by iBGP.

c. Router 1c learns about x from which routing protocol?

Answer: eBGP

- **Reasoning:** Router 1c (in AS1) is connected to Router 3a (in AS3). Since they are in different Autonomous Systems, the routing update is passed via **eBGP**.

d. Router 1d learns about x from which routing protocol?

Answer: iBGP

- **Reasoning:** Router 1d (in AS1) learns the route from its gateway router, 1c. Both routers are in the same Autonomous System (AS1). Therefore, the external route is propagated internally via **iBGP**.